ResumeLens

Formal Language-Based Résumé Screening

Juan Felipe Castro Reyes · Sebastián Mejía

ICESI
Computing and Discrete Structures III (CyED3)
Course project · Semester 2026-2

Problem

- Reading résumés by hand is slow and inconsistent.
- ResumeLens reads a plain-text résumé and decides which job profile fits the candidate.
- It uses only tools from formal language theory.

Four stages, four formal models

1 Extraction — 25 tests
Python re · regular expressions

Regular expressions find contact data, experience, education and skills.

2 Normalization — 29 tests
pyformlang FST · 7-tuple transducer

Each skill, as written, becomes one of 40 standard tokens: `"ReactJS"`, `"React.js"` → **REACT**. Tokens are then sorted in canonical order per profile.

3 Classification — 37 tests
DFA · NFA · ε-NFA · 5-tuple automata

One automaton per job profile accepts or rejects the sorted tokens. A candidate can be accepted by more than one profile.

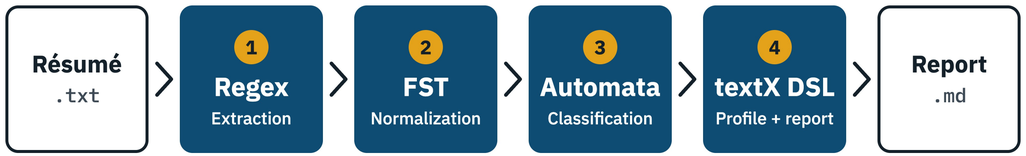
4 DSL — 21 tests
textX · context-free grammar (EBNF)

Describes the final candidate profile in a `.resume` file, validates it and generates a Markdown report.

Design decisions

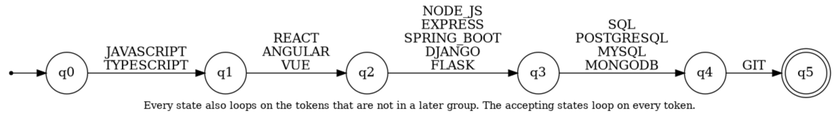
- **Routes.** Each profile is a set of routes made of groups of interchangeable tokens.
- **Order-free.** Tokens are sorted before the automaton reads them, so skill order in the résumé does not change the result.
- **Unknown skills ignored.** Skills outside the vocabulary never block a profile.

Method: a 4-stage pipeline

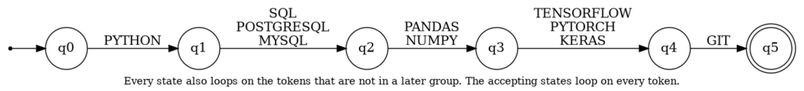


One automaton per profile

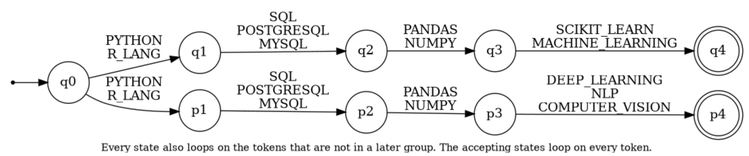
Full Stack Developer · DFA



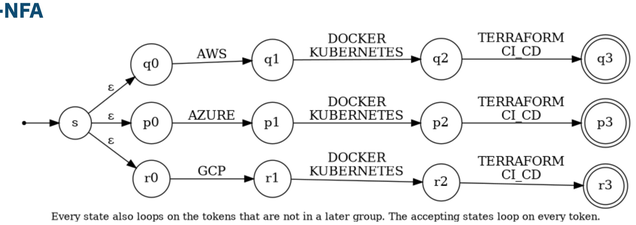
Machine Learning Engineer · DFA



Data Scientist · NFA



Cloud Engineer · ε-NFA



Limitations

- Only skills in the 40-token vocabulary are found.
- Full Stack requires Git.
- The automata depend on the sorted input.

Future work

- Larger vocabulary and more job profiles.
- Support for PDF and Spanish résumés.

Results

Résumé	Tokens (after Stage 2)	Accepted profiles
`fullstack_01`	`JAVASCRIPT, REACT, NODE_JS, POSTGRESQL, GIT`	Full Stack Developer
`fullstack_02`	`JAVASCRIPT, TYPESCRIPT, REACT, NODE_JS, SPRING_BOOT, REST_API, POSTGRESQL, MONGODB, DOCKER`	none (no GIT token)
`mlengineer_01`	`PYTHON, SQL, GIT, PANDAS, NUMPY, SCIKIT_LEARN, TENSORFLOW`	Machine Learning Engineer, Data Scientist
`mlengineer_02`	`PYTHON, SQL, GIT, AWS, PANDAS, SCIKIT_LEARN, TENSORFLOW, PYTORCH, MACHINE_LEARNING, DEEP_LEARNING`	Machine Learning Engineer, Data Scientist
"AWS, Docker, Terraform, Git"	`DOCKER, GIT, AWS, TERRAFORM`	Cloud Engineer

Skills outside the 40-token vocabulary are ignored.

Example: mlengineer_01 through the pipeline

1 · Regex as written — Python, SQL, Git, Pandas, NumPy, Scikit-learn, TensorFlow

2 · FST tokens — `PYTHON, SQL, GIT, PANDAS, NUMPY, SCIKIT_LEARN, TENSORFLOW`
Sorted for ML Engineer: GIT moves to the end
`PYTHON, SQL, PANDAS, NUMPY, SCIKIT_LEARN, TENSORFLOW, GIT`

3 · Automata

ML Engineer (DFA)	Data Scientist (NFA)
ACCEPTED	ACCEPTED
Full Stack	Cloud Engineer
REJECTED	REJECTED

4 · DSL — Classification "Machine Learning Engineer" and "Data Scientist" in the `.resume` file.

134 **automated tests pass**, 23 of them for the pipeline and app.
A Streamlit app shows each stage in its own tab.

Conclusions

- Regex, transducers, automata and a DSL work well together as one pipeline.
- Each stage has a formal model: regular expression, 7-tuple FST, 5-tuple automaton, context-free grammar.

References [1] pyformlang documentation. [2] textX documentation. [3] Streamlit documentation. AI assistance (Claude) used at level 3, see `docs/ai-usage.md`.